

Sequential RISC-V Processor Design in iVerilog

Introduction to Processor Architecture

Spring 2026

Team 33

Gauri Krishnan
2024102073

Dhanika Kothari
2024102011

Moksha Choksi
2024102048

Contents

1	Introduction	2
2	Sequential Datapath Architecture	2
2.1	Program Counter (PC)	3
2.2	Instruction Memory	3
2.3	Register File	4
2.4	Control Unit	5
2.5	Immediate Generation	6
2.6	ALU Control	6
2.7	Arithmetic Logic Unit (ALU)	7
2.8	Data Memory	7
2.9	Multiplexers	8
2.10	Adder Blocks	8
3	The Sequential Processor	8
4	Testing and Verification	8
4.1	Sum of 1st N numbers	8
4.2	Fibonacci Sequence	9
5	Challenges faced	10

1 Introduction

This project focuses on the design and implementation of a Sequential (Non-Pipelined) RISC-V processor based on the RV64I base integer instruction set. The processor was implemented using Verilog HDL and simulated using iVerilog.

The sequential processor executes one complete instruction per clock cycle, meaning that the next instruction begins execution only after the current instruction has fully completed. The supported instructions include:

- Arithmetic: add, sub, addi
- Logical: and, or
- Memory: ld, sd
- Control Flow: beq

The architecture strictly follows a modular design approach, where each component of the datapath was designed, verified independently, and later integrated into the complete processor.

The processor follows the Big-Endian memory organization as specified in the project document.

2 Sequential Datapath Architecture

The sequential processor executes instructions in five logical stages:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

However, unlike pipelined processors, these stages occur sequentially within a single clock cycle. The total clock period must satisfy:

$$T_{clock} \geq T_{IF} + T_{ID} + T_{EX} + T_{MEM} + T_{WB}$$

Thus, the clock frequency is limited by the slowest cumulative path in the datapath.

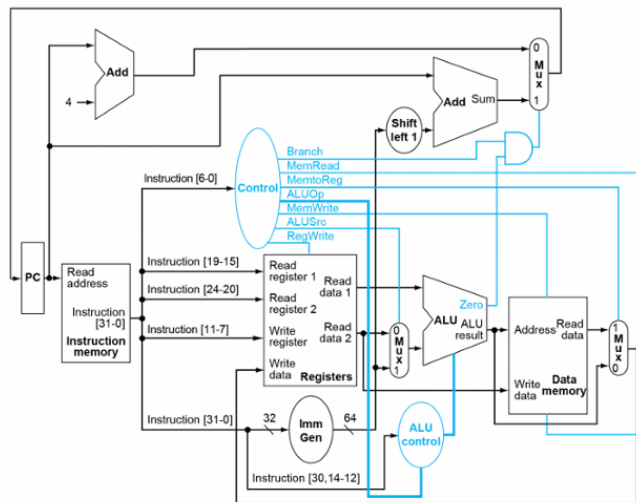


Figure 1: Complete Datapath of the Sequential Processor

2.1 Program Counter (PC)

The Program Counter consists of a 64-bit register that stores the address of the current instruction being executed.

Inputs: clk, reset, pc.in

Outputs: pc.out

The Program Counter is updated on every positive edge of the clock signal. When the reset signal is asserted, the PC is cleared to zero. Otherwise, the value present at pc.in is loaded into the PC.

During normal sequential execution, the PC is incremented by 4 bytes to fetch the next instruction, since each instruction in RV64I is 32 bits (4 bytes) long and memory is byte-addressable. In case of branch or instructions, the PC is updated with the computed branch target address instead of PC + 4.

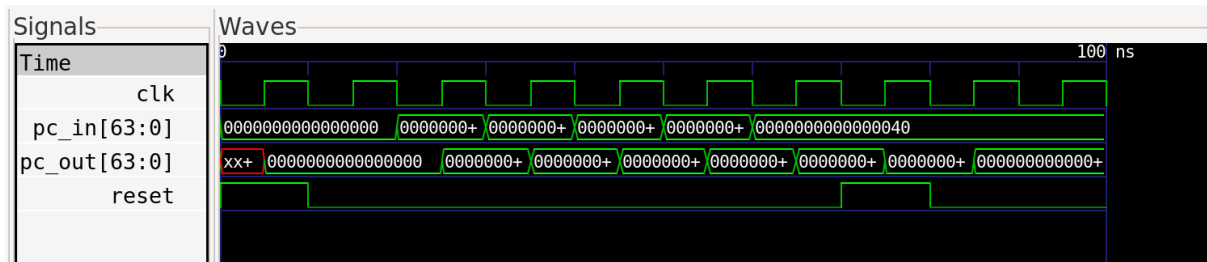


Figure 2: Output of PC file Testbench

2.2 Instruction Memory

The Instruction Memory stores the program instructions fetched by the processor during execution. It is implemented as a byte-addressable memory array and outputs a 32-bit instruction corresponding to the given address.

Inputs: clk, reset, address

Outputs: instruction

A macro IMEM_SIZE defines the total size of the memory block. In this design, a size of 4096 bytes (or greater) is used, which is sufficient for typical program storage.

The instructions are loaded from an external file `instructions.txt` located in the same directory as the Verilog source file. The file contains hexadecimal byte values, with one byte per line. Every four consecutive lines together represent one complete 32-bit instruction.

During simulation initialization, the file is opened and parsed line-by-line. Each byte is stored sequentially into the instruction memory array.

When an address is applied at the input `addr`, the corresponding 32-bit instruction is formed by concatenating four consecutive bytes:

$$instr = \{memory[addr], memory[addr + 1], memory[addr + 2], memory[addr + 3]\}$$

Since each instruction is 32 bits (4 bytes) long, instruction addresses increment by 4 during sequential execution. If the address exceeds the valid memory range, a default value of 32'b0 is returned.

The instruction memory operates as a read-only memory during execution and does not require a write-enable signal.

```

● moksha@LAPTOP-4EPBCM0Q:~/IPA/mpiwithtb/mpi$ iverilog -o instruction_mem_tb instruction_mem.v instruction_mem_tb.v
● moksha@LAPTOP-4EPBCM0Q:~/IPA/mpiwithtb/mpi$ vvp instruction_mem_tb
VCD info: dumpfile instruction_mem_tb.vcd opened for output.
Test 1: Read first instruction
Address = 0000000000000000
Instruction = 00500113 (Expected: 00500113)
Result: PASS

Test 2: Read second instruction
Address = 0000000000000004
Instruction = 00a00193 (Expected: 00a00193)
Result: PASS

Test 3: Boundary check (last valid instruction)
Address = 0000000000000ffc
Instruction = 00000000
Result: PASS

Test 4: Out-of-range address
Address = 0000000000001000
Instruction = 00000000 (Expected: 00000000)
Result: PASS

=====
Total Tests : 4
Passed      : 4
Failed      : 0
=====
instruction_mem_tb.v:120: $finish called at 60000 (1ps)

```

Figure 3: Output for Instruction Memory file Testbench

2.3 Register File

The Register File consists of 32 general-purpose registers, each 64 bits wide.

Inputs: clk, reset, read_reg1, read_reg2, write_reg, write_data, reg_write_en

Outputs: read_data1, read_data2

The register file allows simultaneous reading of two registers and writing to one register in a single clock cycle. The write operation occurs only when reg_write_en is asserted. Register x0 is hardwired to zero and cannot be modified. After execution completes, the contents of all 32 registers are written to `register file.txt` in hexadecimal format.

```

Test 1: Reset verification (all registers zero)
x10 = 0000000000000000 (Expected: 0000000000000000)
Result: PASS

Test 2: Write maximum positive 64-bit value to x5
x5 = 7fffffffffffffff (Expected: 7FFFFFFFFFFFFFFF)
Result: PASS

Test 3: Write negative number to x8
x8 = 18446744073709539271 (Expected: -12345)
Result: PASS

Test 4: Overwrite x5
x5 = 111122233334444 (Expected: 111122233334444)
Result: PASS

Test 5: Simultaneous read of x5 and x8
x5 = 111122233334444
x8 = 18446744073709539271
Result: PASS

=====
Total Tests : 5
Passed      : 5
Failed      : 0
=====
regfile_tb.v:119: $finish called at 65000 (1ps)

```

Figure 4: Output for Register File Testbench

2.4 Control Unit

The Control Unit decodes the opcode field of the instruction and generates the control signals required for proper operation of the datapath.

Input: opcode

Outputs: Branch, MemRead, MemtoReg, ALUOp[1:0], MemWrite, ALUSrc, RegWrite

Based on the instruction type (R-type, I-type, S-type, or B-type), the Control Unit produces the following signals:

- **RegWrite** is asserted for instructions that write a result back to the register file, such as R-type instructions, `addi`, and `ld`. It remains de-asserted for `sd` and `beq`.
- **ALUSrc** selects the second ALU operand. It is set to 1 for instructions using an immediate value (e.g., `addi`, `ld`, `sd`) and set to 0 when both operands come from registers (R-type, `beq`).
- **MemRead** is asserted only for load instructions (`ld`) to enable reading from data memory.
- **MemWrite** is asserted only for store instructions (`sd`) to allow writing to data memory.
- **MemtoReg** determines the source of data written back to the register file. It is set to 1 for load instructions so that memory data is written back, and set to 0 for arithmetic instructions so that the ALU result is written.
- **Branch** is asserted for branch instructions (`beq`). Together with the zero flag from the ALU, it determines whether the PC should update to the branch target address.
- **ALUOp** provides a high-level classification of the instruction to the ALU Control block. One encoding corresponds to R-type operations, another to branch comparisons, and another to immediate-based arithmetic or address calculation.

An indirect control signal is generated by logically ANDing the Branch signal with the zero flag. When asserted, the PC is updated with the branch target address; otherwise, it increments by 4 for sequential execution.

```
Expected -> ALUOp=10, RegWrite=1
Actual   -> ALUOp=10, RegWrite=1

Result : PASS

Test 2 : I-Type (addi)
Expected -> ALUOp=00, ALUSrc=1, RegWrite=1
Actual   -> ALUOp=00, ALUSrc=1, RegWrite=1

Result : PASS

Test 3 : Load (ld)
Expected -> ALUOp=00, MemRead=1, MemtoReg=1
Actual   -> ALUOp=00, MemRead=1, MemtoReg=1

Result : PASS

Test 4 : Store (sd)
Expected -> ALUOp=00, MemWrite=1
Actual   -> ALUOp=00, MemWrite=1

Result : PASS

Test 5 : Branch (beq)
Expected -> ALUOp=01, Branch=1
Actual   -> ALUOp=01, Branch=1

Result : PASS

Test 6 : Undefined Opcode
Expected -> ALUOp=00, All control signals 0
Actual   -> ALUOp=00, Branch=0, RegWrite=0

Result : PASS

=====
Control Unit Verification Summary
Total Cases : 6
Successful   : 6
Unsuccessful : 0
=====
control tb.v:123: $finish called at 30000 (1ps)
```

Figure 5: Output for Control Unit Testbench

2.5 Immediate Generation

The Immediate Generation block extracts and sign-extends the immediate field from the 32-bit instruction.

Input: instruction (32-bit)

Output: immediate (64-bit signed extended)

The immediate is calculated according to the Instruction type as follows:

- For I-type instructions such as addi and ld, the immediate is extracted from bits [31:20].
- For S-type instructions such as sd, the immediate is formed by combining bits [31:25] and [11:7].
- For B-type instructions such as beq, the immediate is constructed using bits [31], [7], [30:25], and [11:8], followed by a left shift of one bit to ensure word alignment. The extracted value is then sign-extended to 64 bits.

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12:10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type

Figure 6: Immediate Reconstruction for each Instruction type

```

Test 1 : I-Type (addi) Positive Immediate
Immediate = 25 | Expected = 25 --> PASS

Test 2 : I-Type (addi) Negative Immediate
Immediate = 18446744073709551600 | Expected = 18446744073709551600 --> PASS

Test 3 : S-Type (sd)
Immediate = 8 | Expected = 8 --> PASS

Test 4 : B-Type (beq) Positive Offset
Immediate = 16 | Expected = 16 --> PASS

Test 5 : B-Type (beq) Negative Offset
Immediate = 18446744073709551608 | Expected = 18446744073709551608 --> PASS

=====
Immediate Generator Verification
Total Tests : 5
Passed      : 5
Failed      : 0
=====
imm_gen_tb.v:106: $finish called at 25000 (1ps)

```

Figure 7: Output for Immediate Generator Testbench

2.6 ALU Control

The ALU Control block determines the exact ALU operation to be performed.

Inputs: ALUOp[1:0], funct3, funct7

Output: ALUControl (4-bit)

The ALUOp signal from the Control Unit indicates the general instruction type, while the funct3 and funct7 fields further specify the exact operation for R-type instructions. Based on these inputs, the ALUControl signal selects the required arithmetic or logical function such as addition, subtraction, AND, or OR.

```

===== Test 1 : ALUOp=00 (Load/Store/Addi) =====
Expected = 0000 | Actual = 0000 | PASS

===== Test 2 : ALUOp=01 (Branch) =====
Expected = 1000 | Actual = 1000 | PASS

===== Test 3 : R-Type ADD =====
Expected = 0000 | Actual = 0000 | PASS

===== Test 4 : R-Type SUB =====
Expected = 1000 | Actual = 1000 | PASS

===== Test 5 : R-Type AND =====
Expected = 0111 | Actual = 0111 | PASS

===== Test 6 : R-Type OR =====
Expected = 0110 | Actual = 0110 | PASS

===== Test 7 : Invalid funct3 =====
Expected = 1111 | Actual = 1111 | PASS

===== Test 8 : Invalid ALUOp =====
Expected = 1111 | Actual = 1111 | PASS

=====
ALU Control Verification Summary
Total Cases : 8
Successful   : 8
Unsuccessful: 0
=====
alu_control_tb.v:97: $finish called at 40000 (1ps)

```

Figure 8: Output for ALU Control Testbench

2.7 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit performs arithmetic and logical operations on two 64-bit operands.

Inputs: input1 (64-bit), input2 (64-bit), control

Outputs: result (64-bit), zero flag

Depending on the control signal, the ALU performs operations such as addition, subtraction, AND, and OR. The zero flag is asserted when the result is equal to zero, which is used in evaluating branch conditions.

2.8 Data Memory

The Data Memory is a byte-addressed memory block of size 1024 bytes.

Inputs: clk, reset, address, write_data, MemRead, MemWrite

Output: read_data (64-bit) When MemWrite is asserted, the 64-bit value present on write_data is

written to the specified memory address. When MemRead is asserted, the data stored at the specified address is returned on read_data. The memory has a latency of one clock cycle, meaning the output data becomes available at the next positive clock edge. The contents of data memory are written to `data mem.txt` after execution.

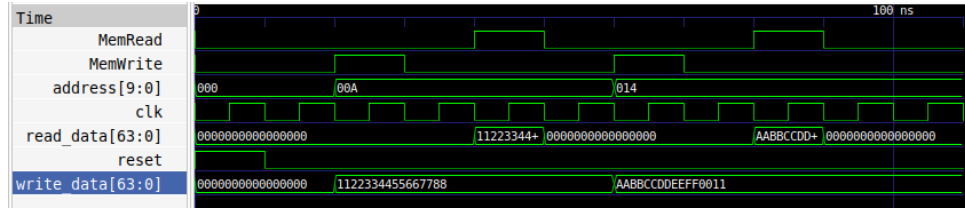


Figure 9: GTK-Wave Output for Data Memory Testbench

2.9 Multiplexers

Multiplexers are used at multiple points in the datapath to select between alternative inputs.

Inputs and Outputs: Depend on placement in datapath as follows:

- One multiplexer selects between register data and immediate value for the second ALU operand, controlled by ALUSrc.
- Another multiplexer selects between the ALU result and data memory output during the write-back stage, controlled by MemtoReg.
- A third multiplexer selects between $PC + 4$ and the branch target address, controlled by the logical AND of the Branch signal and the zero flag.

2.10 Adder Blocks

Two adder blocks are used in the datapath.

Inputs and Outputs: 64-bit operands, 64-bit sum

One adder computes $PC + 4$ for sequential execution. The other adder computes the branch target address by adding the sign-extended immediate to the current PC value.

3 The Sequential Processor

The individual modules were integrated into a wrapper file to form the complete Sequential Processor. The processor executes one instruction per clock cycle and runs for an extra clock cycle to detect the end of the program. This implementation supports the following RISC-V instructions: addi, and, sub, add, or, ld, sd, and beq.

4 Testing and Verification

Each individual module was tested using verilog testbenches and the final Sequential Processor was tested using assembly code that was converted to machine language instructions.

The assembly code and the machine language instructions are saved as `name_instructions_exp.txt` and `name_instructions.txt` respectively.

The final output is stored in `register_file.txt` and contains the 32 64-bit register values in hexadecimal, followed by the number of clock cycles in decimal.

4.1 Sum of 1st N numbers

Assembly code:

```

1 addi x1, x0, 30 //value of N
2 addi x2, x0, 0
3 addi x3, x0, 1
4 add x10, x3, x0
5 add x2, x2, x3
6 addi x3, x3, 1
7 sub x4, x3, x1

```

```

8 beq x4, x10, 8
9 beq x0, x0, -16

```

Output in register_file.txt:

```

x0: 0000000000000000
x1: 000000000000001e // N = 30
x2: 00000000000001d1 // sum
x3: 000000000000001f // value of i after exit
x4: 0000000000000001
x5: 0000000000000000
x6: 0000000000000000
x7: 0000000000000000
x8: 0000000000000000
x9: 0000000000000000
x10: 0000000000000001
Cycle count: 154

```

4.2 Fibonacci Sequence

Assembly code:

```

1 addi x1, x0, 8
2 addi x2, x0, 0
3 addi x3, x0, 0
4 addi x4, x0, 1
5
6 sd x3, 0(x2)
7 addi x2, x2, 8
8 sd x4, 0(x2)
9 addi x2, x2, 8
10
11 addi x5, x0, 2
12 beq x5, x1, 32
13
14 loop:
15 add x6, x3, x4
16 sd x6, 0(x2)
17 addi x2, x2, 8
18 addi x5, x5, 1
19 add x3, x0, x4
20 add x4, x0, x6
21 beq x5, x1, 12
22 beq x0, x0, -28
23
24 //load
25 addi x2, x0, 0
26 ld x10, 0(x2)
27 ld x11, 8(x2)
28 ld x12, 16(x2)
29 ld x13, 24(x2)
30 ld x14, 32(x2)
31 ld x15, 40(x2)
32 ld x16, 48(x2)
33 ld x17, 56(x2)

```

Output in register_file.txt:

```

x0 : 0000000000000000
x1 : 0000000000000008 // loop limit (no. of terms)
x2 : 0000000000000000

```

```
x3 : 000000000000000008
x4 : 00000000000000000d
x5 : 000000000000000008 // value of i after exit
x6 : 00000000000000000d // last computed value
x7 : 000000000000000000
x8 : 000000000000000000
x9 : 000000000000000000
x10 : 000000000000000000 //0
x11 : 000000000000000001 //1
x12 : 000000000000000001 //1
x13 : 000000000000000002 //2
x14 : 000000000000000003 //3
x15 : 000000000000000005 //5
x16 : 000000000000000008 //8
x17 : 00000000000000000d //13
Cycle count: 70
```

5 Challenges faced

- Consistent implementation of Big-endian memory format throughout the project
- Implementing accurate immediate reconstruction according to each Instruction type
- Proper byte addressing for 64-bit load/store operations
- Correct handling of branch termination condition
- Stopping execution exactly when a 0 instruction is fetched without missing or adding extra cycles.